Administrarea bazelor de date

PL/SQL - CURS 3: Exceptii (2); Tipurile RECORD si TABLE ... INDEX BY

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Domeniul de valabilitate excepţii

- O excepţie definită într-un bloc este locală blocului respectiv şi globală pentru toate subblocurile acestuia (dacă nu este redefinită), dar nu este cunoscută în afara blocului în care e declarată.
- În cazul în care un bloc tratează excepţiile apărute, el se termină fără eroare şi procesarea continuă cu instrucţiunile care urmează după END-ul blocului respectiv.

Domeniul de valabilitate excepţii

- În cazul în care însă un bloc nu tratează o anumită excepţie în propria sa zonă EXCEPTION aceasta se propagă la blocurile înconjurătoare până găseşte tratarea adecvată.
- Exemplu: Două excepţii cu acelaşi nume sunt definite în bloc şi în subbloc. Ele sunt considerate diferite: prima aparţine blocului fără subbloc iar a doua doar subblocului.
- În cazul urmator excepţia ridicată în subbloc nu este tratată de blocul înconjurător.

Exemplu

```
DECLARE
exceptia1 EXCEPTION;
BEGIN
   DECLARE -- incepe un subbloc
   exceptia1 EXCEPTION; -- redeclarare in subbloc
   BEGIN
       ...
       IF ... THEN
             RAISE exceptia1; -- nu e tratata de bloc
             -- fiind redefinita in subbloc
       END IF;
   END; -- sfarsit subbloc
EXCEPTION
WHEN exceptia1 THEN -- nu trateaza exceptia din subbloc
...
END;
```

Domeniul de valabilitate excepţii

- Pentru a trata excepţia apărută în subbloc se poate proceda în mai multe moduri:

1. Se elimină declaraţia excepţiei din subbloc în care caz ea va fi tratată de clauza WHEN existentă în bloc
2. Se adaugă o zonă EXCEPTIONS în subbloc în care să se trateze această excepţie, tratare care poate fi diferită de cea existentă în bloc.
3. Se adaugă WHEN OTHERS la excepţiile blocului în care caz toate excepţiile netratate în subbloc şi bloc vor putea fi tratate aici.

RAISE_APPLICATION_ERROR

- Excepţiile de acest tip sunt similare erorilor Oracle non-predefinite dar trebuiesc ridicate explicit în zona executabilă sau în zona de excepţii folosind (sunt erori definite de user):

```
RAISE_APPLICATION_ERROR(cod_eroare, text_eroare)
```

unde:

- **cod_eroare** este un număr ales de utilizator între -20000 şi -20999
- **text_eroare** este un text ales de utilizator

- Şi la aceste excepţii se poate folosi PRAGMA EXCEPTION_INIT pentru a le asigna un nume!

Exemplu

```
DECLARE
   v_suma   NUMBER;
   suma_nula      EXCEPTION;
   PRAGMA EXCEPTION_INIT(suma_nula, -20021);
BEGIN
   SELECT suma INTO v_suma
   FROM bursa
   WHERE pmin < 100;
   IF v_suma IS NULL THEN
       RAISE_APPLICATION_ERROR(-20021, 'Suma este nula');
   END IF;
EXCEPTION
   WHEN suma_nula THEN
         dbms_output.put_line(SQLERRM);
END;
```

RAISE_APPLICATION_ERROR

- În cazul în care nu are asociat un nume, excepţia se poate trata în zona WHEN OTHERS folosind SQLCODE:

Exemplu

```
DECLARE
   v_suma   NUMBER;
   v_coderoare NUMBER;
BEGIN
   SELECT suma INTO v_suma
   FROM bursa WHERE pmin < 100;
   IF v_suma IS NULL THEN
      RAISE_APPLICATION_ERROR(-20021, 'Suma este nula');
   END IF;
EXCEPTION
   WHEN OTHERS THEN
      v_coderoare := SQLCODE;
      IF (v_coderoare = -20021) THEN
                dbms_output.put_line(SQLERRM);
      END IF;
END;
```

Retratarea excepţiilor

- În cazul în care se doreşte tratarea unei excepţii apărute într-un subbloc atât în acesta cât şi în blocul înconjurător, se procedează în modul următor:
  - Se prevede tratarea excepţiei respective în zona EXCEPTIONS a subblocului.
  - La sfârşitul secvenţei de instrucţiuni care tratează excepţia se plasează instrucţiunea RAISE fără parametri. Ea propaga excepţia curentă spre blocul înconjurător.
  - Se prevede tratarea excepţiei respective în zona EXCEPTIONS a blocului.

Exemplu

```
DECLARE
   exceptia1 EXCEPTION;
BEGIN
   DECLARE -- incepe un subbloc
   BEGIN
   ...
   IF ... THEN
        RAISE exceptia1; -- nu e tratata de bloc
                                -- fiind redefinita in subbloc
   END IF;
   EXCEPTION -- tratare exceptii subbloc
        WHEN exceptia1 THEN
        -- instructiuni tratare in subbloc
        .  .  .  .
        RAISE;
   END; -- sfarsit subbloc
EXCEPTION
WHEN exceptia1 THEN -- retratare exceptia1 in bloc
...
END;
```

Notă: RAISE fără parametri poate sa apara doar în zona de tratare a excepţiei!

Excepţii în declaraţii

- Excepţiile care apar în zona DECLARE determină terminarea execuţiei blocului curent şi nu pot fi tratate în zona EXCEPTIONS a acestuia.
- În schimb ele pot fi tratate în blocul înconjurător (dacă există)!
- Un exemplu de astfel de excepţie este iniţializarea necorespunzătoare a unei variabile (ca mai jos unde într-o constantă de 3 cifre se încearcă stocarea unui număr de 4 cifre)

```
sal_minim CONSTANT NUMBER(3) := 1200;
```

Excepţii în tratarea excepţiilor

- În zona de tratare a excepţiilor se poate ridica aceeaşi excepţie (cum a fost descris mai sus) sau o altă excepţie.
- În toate cazurile excepţia respectivă poate fi tratată doar în blocul înconjurător şi nu în blocul curent

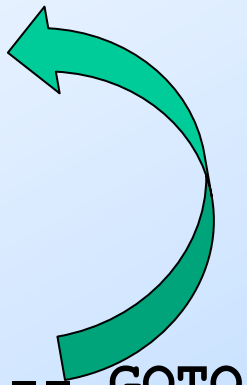
GOTO

- Instrucţiunea GOTO nu poate fi folosită pentru a se trece din zona de tratare a excepţiilor în zona executabilă a blocului respectiv şi nici invers:

```
GOTO <<eticheta>>
```

- Exemplu de GOTO ilegal:

```
BEGIN
<<o_eticheta>>
SELECT .   .   .;
EXCEPTION
WHEN . . . THEN
GOTO <<o_eticheta>>; -- GOTO ilegal
END;
```



- Ea poate fi folosită însă pentru salt din zona de tratare a excepţiilor în zona executabilă a unui bloc înconjurător!

Toate instrucţiunilor blocului

- În cazul în care apare o excepţie, restul instrucţiunilor blocului nu se mai execută.
- Dacă se doreşte execuţia acestora şi în cazul apariţiei unei excepţii, instrucţiunile care cauzează astfel de eveniment trebuiesc transformate în subblocuri.
- Să considerăm următorul bloc:

```
DECLARE
.  .  .
BEGIN
  INSERT . . . ;
  UPDATE . . . ;
  SELECT . . . ;
EXCEPTION
. . .
END;
```

Toate instrucţiunilor blocului

- În cazul apariţiei unei excepţii la INSERT următoarele două cereri SQL nu se mai execută.
- La fel, în cazul apariţiei unei excepţii la UPDATE nu se mai execută SELECT-ul.
- Soluţia este includerea fiecarei astfel de instructiuni intr-un subbloc care isi trateaza exceptiile.
- Daca apare o eroare, ea este tratata si subblocul se termina cu succes. In acest fel se executa toate instructiunile blocului.

Exemplu

```
DECLARE
.  .  .
BEGIN
   BEGIN -- subbloc pentru INSERT
        INSERT . . . ;
   EXCEPTION -- exceptii subbloc INSERT
       WHEN OTHERS THEN
            . . .
   END; -- end subbloc INSERT
   BEGIN -- subbloc pentru UPDATE
        UPDATE . . . ;
   EXCEPTION -- exceptii subbloc UPDATE
       WHEN OTHERS THEN
            . . .
   END; -- end subbloc UPDATE
   SELECT . . . ;
EXCEPTION -- exceptii pentru bloc
. . .
END; -- end bloc
```

Azi: TIPURI COMPUSE:

- RECORD
- TABLE .. INDEX BY

Declarare

- Pentru descrierea tipurilor compuse se poate folosi declaraţia de tip. Aceasta are următoarea formă:

```
TYPE denumire_tip descriere_tip;
```

- Cu ajutorul ei vom putea defini şi tipurile compuse care se pot folosi în PL/SQL:

1. Înregistrare (RECORD)
2. Tablou asociativ (TABLE .. INDEX BY)
3. Tabelă (Nested TABLE)
4. Tablou de dimensiune variabilă (VARRAY)

Declarare

- Ultime trei tipuri de date compuse de mai sus sunt denumite generic ***colecţii***.
- Acest curs prezintă primele doua tipuri compuse: înregistrările şi tablourile asociative.

Înregistrări (RECORD)

- Înregistrările din PL/SQL sunt similare înregistrărilor din limbajul Pascal sau structurilor din C.
- Sub un acelaşi nume sunt înmagazinate mai multe valori de tipuri diferite care se numesc ***câmpuri***.

Înregistrări (RECORD)

- Definirea unei înregistrări se poate face în două moduri:
  - Folosind descrierea fiecărei componente a înregistrării. În acest caz o variabilă de acest tip poate să nu fie asociată cu o tabelă sau cu rezultatul unei cereri SQL.
  - Folosind %ROWTYPE se poate defini o înregistrare compatibilă cu o linie a unei tabele sau cu o linie a rezultatului unei cereri SQL.

Descrierea componentelor

- În acest caz sintaxa definirii unui tip este următoarea:

```
TYPE denumire_tip IS RECORD
(nume_camp tip_camp [NOT NULL]
                    [:= | DEFAULT
  expresie]
[, nume_camp ...]);
```

Descrierea componentelor

- Câmpurilor li se poate asocia o valoare iniţială (prin **:=** sau **DEFAULT**) şi se poate specifica faptul că nu sunt permise valori nule (**NOT NULL**).
- Tipul unui câmp poate fi oricare dintre tipurile scalare sau compuse din PL/SQL în afară de REF CURSOR.
- Referirea unui câmp al unei variabile de tip înregistrare se face prin construcţia:

```
nume_inregistrare.nume_camp
```

Exemplu

```
DECLARE
  v_punctaj              NUMBER(4);
  TYPE t_student IS RECORD
      (matr       NUMBER(4) NOT NULL := 1234,
      nume        stud.nume%type,
      punctaj     v_punctaj%type,
      data        DATE DEFAULT SYSDATE);
  v_student t_student;

BEGIN
  v_student.matr := v_student.matr + 1;
  v_student.nume := 'MARCEL';
  v_student.punctaj := 1400;
  .   .   .
END;
```

Exemplu

- Mai sus a fost definit un tip de înregistrare numit ***t_student*** şi o variabilă de acest tip (***v_student***).
- Se observă că se poate defini tipul unui câmp şi în funcţie de tipul unei variabile sau al unei coloane a unei tabele.
- În exemplul de mai sus toate câmpurile înregistrării sunt scalare dar în cazul general ele pot fi şi compuse (de exemplu înregistrare, definită inclusiv cu %ROWTYPE, descris în continuare).

Definite cu %ROWTYPE

- În acest caz sintaxa definiţiei unei variabile înregistrare este:

```
nume_variabila   nume_tabela%ROWTYPE;
```

- Înregistrarea va avea atâtea câmpuri câte coloane are tabela şi acestea se vor numi la fel cu aceste coloane.
- Astfel de înregistrări se pot utiliza pe clauza INTO a unei cereri SELECT * care întoarce o singură linie.

Exemplu

```
DECLARE
  v_student stud%ROWTYPE;
BEGIN
  SELECT *   INTO v_student
  FROM stud  WHERE matr=1456;
  v_student.nume := 'MARCEL';
  v_student.punctaj := 1400;
  .   .   .
END;
```

Tablouri asociative (TABLE . . . INDEX BY)

- Tablourile asociative stochează perechi (cheie, valoare) unde valorile sunt de acelaşi tip.
- Nu au dimensiune fixată la declarare şi fiecare componentă se creează la prima asignare.
- Caracteristici:

1. Când sunt numerice, cheile nu e obligatoriu să fie consecutive şi nici să înceapă de la 1.
- În Oracle8 cheia putea fi doar BINARY_INTEGER, dar începând cu Oracle9 ea poate fi şi VARCHAR2, un subtip al VARCHAR2 sau un tip convertibil la VARCHAR2.

Tablouri asociative (TABLE . . . INDEX BY)

- Caracteristici:

2. În Oracle8 valorile puteau fi scalare sau de tip ROWTYPE.
- Începând cu Oracle9 poate fi orice tip definit anterior.
3. Elementele unei astfel de colecţii nu pot fi iniţializate la declarare.

Tablouri asociative (TABLE . . . INDEX BY)

- Caracteristici:

4. Tablourile asociative sunt concepute pentru a stoca date temporare ale programelor PL/SQL.
- De aceea ele nu pot fi utilizate în comenzi SQL ca SELECT sau INSERT pentru a fi regăsite/stocate în baza de date.
5. Din punct de vedere al echivalenţei cu structuri de date folosite în alte limbaje de programare, ele sunt similare tabelelor de dispersie (hash tables).

Tipuri admise

- Tipuri admise pentru cheie:
  - BINARY_INTEGER sau PLS_INTEGER.
  - VARCHAR2(n) sau subtipurile sale: VARCHAR, STRING, LONG.
  - Tipuri convertibile la VARCHAR2 cum sunt DATE, TIMESTAMP.
- Tipuri care nu sunt admise pentru cheie:
  - RAW, LONG RAW, ROWID
  - CHAR, CHARACTER.

Declarare

- Sintaxa declarării unui tablou asociativ:

```
TYPE nume_tip IS TABLE OF
     tip_element [NOT NULL]
INDEX BY [BINARY_INTEGER |
          PLS_INTEGER |
          VARCHAR2(dim_max)];
```

Metode

- Colecţiile au asociate o serie de ***metode***.
- Acestea sunt proceduri şi funcţii puse la dispoziţie de sistem pentru orice variabilă de acest tip iar apelul lor se face prin construcţia:

```
variabila.metoda[(parametri)]
```

- In cazul tablourilor asociative se pot folosi următoarele metode:

Metode

Metoda	Efect
`EXISTS(n)`	TRUE dacă elementul cu cheia ***n*** există (este setat)
`COUNT`	Numărul de elemente din tablou.
`FIRST, LAST`	Valoarea **cheii** pentru primul, respectiv ultimul element din tablou.
`PRIOR(n),` **`NEXT(n)`**	Valoarea **cheii** care precede respectiv succede în tablou elementul cu cheia ***n***. Dacă nu există un astfel de element întoarce NULL.
`DELETE[(n[,k)]]`	DELETE: şterge toate **elementele** tabloului. DELETE(n) şterge elementul cu cheia ***n*** DELETE(n, k) şterge toate elementele cu cheia în plaja ***n . . k***

Exemplu

```
DECLARE
TYPE t_varsta IS TABLE OF NUMBER INDEX BY VARCHAR2(30);
TYPE t_nrstud IS TABLE OF NUMBER INDEX BY BINARY_INTEGER;
TYPE t_stud IS TABLE OF stud%ROWTYPE
  INDEX BY BINARY_INTEGER;
v_varsta   t_varsta;
v_nrstud   t_nrstud;
v_stud     t_stud;
BEGIN
v_varsta('Ionescu') := 23;
v_varsta('Badea') := 25;
v_varsta('Avramescu') := 27;
```

Exemplu

```
dbms_output.put_line(v_varsta.FIRST);
-- Avramescu
dbms_output.put_line(v_varsta.LAST);
-- Ionescu
v_nrstud(1211) := 48; -- Indicii nu incep de la 1
v_nrstud(1221) := 29;
v_nrstud(1231) := 20;
v_nrstud(1241) := 21;
v_nrstud(1251) := 17; -- sunt 5 componente
v_stud(4875).nume := 'MARCEL';
v_stud(3456).an   := 1;
v_stud(3456).punctaj  := 1400;  -- 2 componente
END;
```

Exemplul 2

```
DECLARE
TYPE t_nrstud IS TABLE OF NUMBER INDEX BY
  BINARY_INTEGER;
TYPE t_stud IS TABLE OF stud%ROWTYPE INDEX BY
  BINARY_INTEGER;
v_nrstud          t_nrstud;
v_stud             t_stud;
BEGIN
  v_nrstud(1211) := 48;  -- Cheile nu incep de la 1
  v_nrstud(1221) := 29;
  v_nrstud(1231) := 20;
  v_nrstud(1241) := 21;
  v_nrstud(1251) := 17;              -- 5 componente
```

Exemplul 2

```
dbms_output.put_line('FIRST: ' || v_nrstud.FIRST);
  dbms_output.put_line('LAST: ' ||v_nrstud.LAST);
  dbms_output.put_line('COUNT: ' ||v_nrstud.COUNT);
if v_nrstud.EXISTS(1200) then
  dbms_output.put_line('Exista cheia 1200');
else
  dbms_output.put_line('Nu exista cheia 1200');
end if;
if v_nrstud.EXISTS(1231) then
  dbms_output.put_line('Exista cheia 1231');
else
  dbms_output.put_line('Nu exista cheia 1231');
end if;
```

Exemplul 2

```
dbms_output.put_line('PRIOR(1241): '
  ||v_nrstud.PRIOR(1241));
dbms_output.put_line('PRIOR(1211): '
  ||v_nrstud.PRIOR(1211));
    -- valoarea lui PRIOR este NULL
dbms_output.put_line('NEXT(1241): '
  ||v_nrstud.NEXT(1241));
dbms_output.put_line('NEXT(1251): '
  ||v_nrstud.NEXT(1251));
    -- valoarea lui NEXT este NULL
v_nrstud.DELETE(1);     -- cheia nu exista
dbms_output.put_line('Stergere element cu cheia
  1. Raman:' ||v_nrstud.COUNT|| ' elemente');
```

Exemplul 2

```
v_nrstud.DELETE(1241);  -- cheie existenta
dbms_output.put_line('Au ramas: '||v_nrstud.COUNT||'
  elemente');
v_nrstud.DELETE(1220, 1300);  -- sterge chei in
                              -- plaja data
dbms_output.put_line('Au ramas: '||v_nrstud.COUNT||'
  elemente');
v_stud(4875).nume := 'MARCEL';
v_stud(3456).an   := 2;
v_stud(3456).punctaj  := 2400;
                -- sunt 2 chei distincte
dbms_output.put_line('COUNT: '||v_stud.COUNT);
                -- afiseaza 2
END;
```

Exemplul 2

- Rezultatul execuţiei este:

```
FIRST: 1211
LAST: 1251
COUNT: 5
Nu exista cheia 1200
Exista cheia 1231
PRIOR(1241): 1231
PRIOR(1211):
NEXT(1241): 1251
NEXT(1251):
Stergere element cu cheia 1. Raman:5 elemente
Au ramas: 4 elemente
Au ramas: 1 elemente
COUNT: 2
```

Sfârşitul cursului 3